

Structure du Programme (Extrait de Code)

Projet : Voltmètre / Fréquencemètre

1. La Boucle Principale

```
void loop() {  
    bool etatBouton = digitalRead(boutonPin);    // Lecture de l'état actuel du bouton (HIGH ou LOW)  
    if (etatBouton == LOW && etatBoutonPrec == HIGH) {    // Détection si le bouton est appuyé  
        page = (page + 1) % 3;                // Incrémente la page. Le modulo 3 permet de revenir à 0 après 2  
        lcd.clear();  
        delay(200);  
    }  
    etatBoutonPrec = etatBouton;                // Mémorise l'état actuel pour le prochain tour de boucle  
  
    if (page == 0 || page == 1) {                // Si on est sur les pages Voltmètre (0 ou 1) => calcul de tension  
        calculerVoltmetre();  
    } else {                                    // Sinon (page 2) => calcul de fréquence  
        calculerFrequence();  
    }  
  
    if (millis() - tDernierAffichage > 300) {    // Vérifie si 300ms se sont écoulées depuis le dernier affichage  
        afficherLCD();                        // Met à jour l'écran  
        tDernierAffichage = millis();          // Redémarre le chronomètre d'affichage  
    }  
}
```

2. Les fonctions de calcul

A. Fonction voltmètre

```
void calculerVoltmetre() {                    // Fonction pour le voltmètre  
    int mesureNum = analogRead(pinVoltmetre);    // Lecture analogique (0 à 1023)  
  
    // Tension instantannée  
    tensionInstantanee = mesureNum * (5.0 / 1023.0); // Conversion produit en croix vers des Volts (0 à 5V)  
  
    // Vmoy  
    somme -= mesure[nb];                        // On retire l'ancienne valeur stockée à cet index de la somme totale  
    mesure[nb] = tensionInstantanee;            // On remplace par la nouvelle mesure  
    somme += mesure[nb];                        // On ajoute la nouvelle mesure à la somme totale  
    nb = (nb + 1) % NB_ECH;                    // On avance l'index d'une case (et on revient à 0 si on dépasse 99)
```

```

    valeurMoyenne = somme / NB_ECH;          // Calcul de la moyenne

    //Veff
    float sommeCarres = 0.0;                  // Somme des carrés
    for (int i = 0; i < NB_ECH; i++) {       // Parcours entier du tableau
        sommeCarres += mesure[i] * mesure[i]; // Addition des carrés de chaque tension
    }
    valeurRMS = sqrt(sommeCarres / NB_ECH);  // Calcul de la tension efficace

    //Vac
    float varTemp = valeurRMS * valeurRMS - valeurMoyenne * valeurMoyenne; // Calcul de la composante
    alternative seule au carré
    if (varTemp < 0) varTemp = 0;            // Sécurité pour éviter racine d'un nombre négatif
    valeurRMS_AC = sqrt(varTemp);            // Calcul de la composante alternative seule

    delay(100);                             // Delay pour lisser l'évolution des valeurs à l'écran
}

```

B. Fonction fréquencemètre

```

void calculerFrequence() {                  // Fonction pour le fréquencemètre
    long timeout = 2000000;                 // Sécurité qui permet de sortir de la boucle while au bout de timeout µs
    unsigned long t1, t2;                   // Variables pour stocker les temps (en microsecondes)

    unsigned long startWait = micros();     // On lance un chronomètre de sécurité

    while(analogRead(pinGBF) > 500) {       // Attend que le signal soit en BAS (en dessous de ~2.44V)
        if (micros() - startWait > timeout) {
            frequence = 0.0;
            periode_ms = 0.0;
            return;                         // Si trop long, on quitte la fonction
        }
    }

    while(analogRead(pinGBF) < 520) {        // Attend que le signal soit en HAUT (en dessous de ~2.54V)
        // (Utilisation de deux seuils (500 et 520) pour éviter le bruit (Hystérésis logicielle))
        if (micros() - startWait > timeout) {
            frequence = 0.0;

```

```

    periode_ms = 0.0;

    return;                // Si trop long, on quitte la fonction
}
}

t1 = micros();            // On mémorise l'heure exacte du premier passage par 520 (début de
                           période)

while(analogRead(pinGBF) > 500) {    // Attend que le signal repasse en dessous de 500
    if (micros() - startWait > timeout) {
        frequence = 0.0;
        periode_ms = 0.0;

        return;                // Si trop long, on quitte la fonction
    }
}

while(analogRead(pinGBF) < 520) {    // Attend que le signal repasse au dessus de 520 (Une période
complète)
    if (micros() - startWait > timeout) {
        frequence = 0.0;
        periode_ms = 0.0;

        return;                // Si trop long, on quitte la fonction
    }
}

t2 = micros();                // On mémorise l'heure exacte du second passage par 520

unsigned long periode_us = (t2 - t1);    // Différence de temps en microsecondes
if (periode_us > 0) {            // Sécurité qui vérifie que la période est positive
    frequence = 1000000.0 / periode_us;    // f = 1 / T (avec conversion µs en secondes)
    periode_ms = periode_us / 1000.0;    // Conversion en millisecondes pour l'affichage
}
}

```